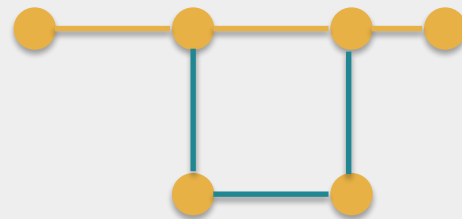


Version Control

with Git & GitHub

A practical introduction for research and collaborative coding



The problem: collaborating without version control

Benta and Fatou need to work on the same report at the same time.

Option A — take turns

Safe, but slow: one person waits while the other edits.

Option B — email copies

Fast at first, but changes can be duplicated, overwritten, or lost.



Version control gives them a shared history without forcing them to work sequentially.

Learning outcomes

By the end of the session, you should be able to:

1

Explain why version control is useful for research and code projects

4

Create or clone a repository and inspect its state

2

Distinguish Git from GitHub

5

Use add, commit, log, push and pull in a simple workflow

3

Explain repository, working directory, staging area, commit, branch, local and remote

6

Use branches for collaboration and identify files that should not be tracked

Types of VCS

Local Version Control

Each person keeps a complete history of the project on their own computer.

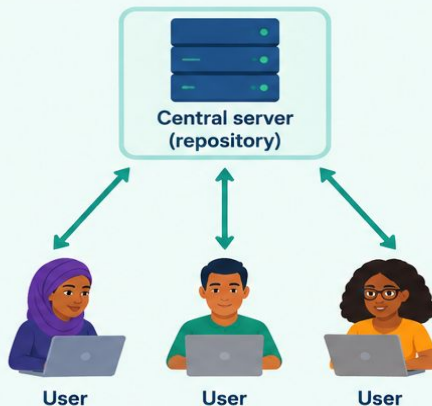


Key idea:

Works offline, but changes are not shared with others unless you manually copy them.

Centralized Version Control

A central server stores the full history. Everyone works with the same central repository.

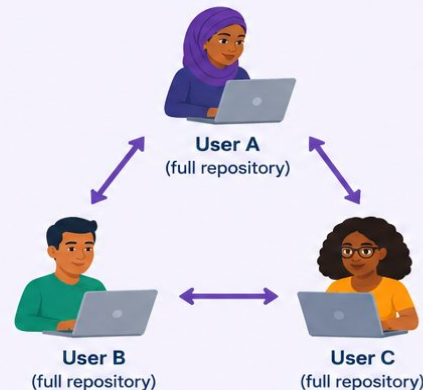


Key idea:

All changes go through the central server, which can be a single point of failure.

Distributed Version Control

Each person has a full copy of the repository, including the complete history. Changes are shared directly between peers (and can also be pushed to a central server).



Key idea:

Everyone has a full copy and can work offline. Changes are shared directly between peers (and can be synced with a central server).

What version control gives you

Think of it as a structured history of meaningful project changes.

1

History

Inspect how a file or project changed over time.

2

Recovery

Return to an earlier known-good version.

3

Attribution

See who changed what, when, and why.

4

Collaboration

Combine independent work transparently.

The “final file” problem

Filenames are a poor substitute for a version history.

Name
 Press release for approval.doc
 Press release final.doc
 Press release FINAL VERSION.doc
 Press release FINAL FINAL VERSION.doc
 IMPROVED FINAL PRESS RELEASE.doc
 REVISED APPROVED FINAL PRESS RELEASE.doc
 REVISED APPROVED FINAL PRESS RELEASE v. 2.doc
 !! NEW REVISED APPROVED FINAL PRESS RELEASE v. 2.doc
 !!! REVISED NEW REVISED APPROVED FINAL PRESS RELEASE v. 2.doc
 !!!! Press release as sent.doc

Instead of encoding history in filenames...

```
commit 7a2f  Add methods section
commit 9c14  Fix missing-value
             handling
commit e31b  Update Figure 2
```

**A commit history is ordered,
attributable, and searchable.**

Git is not GitHub

They work together, but they solve different parts of the problem.

Git

- Version-control software
- Runs on your computer
- Works offline
- Tracks local project history
- Commands: add, commit, log...

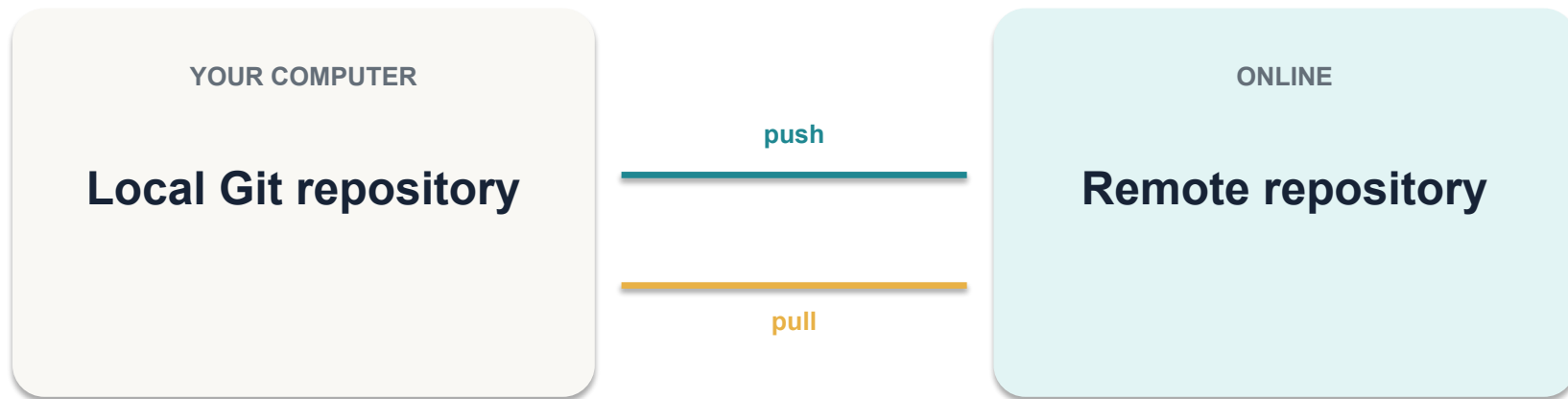
GitHub

- Online repository hosting platform
- Supports remote collaboration
- Stores a remote copy of a Git repository
- Adds pull requests, issues and review

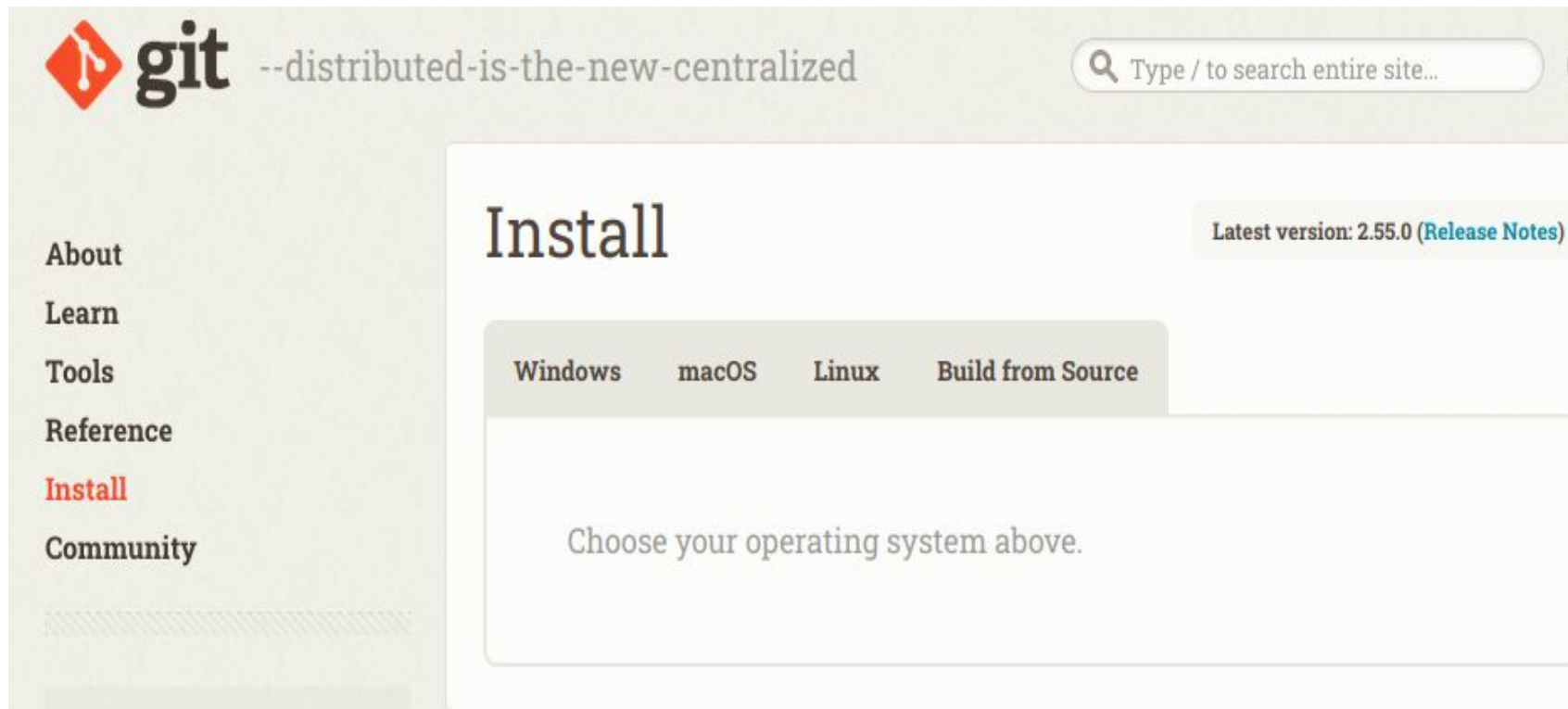
You can use Git without GitHub.

Local and remote repositories

Git works locally; a remote lets collaborators exchange history.

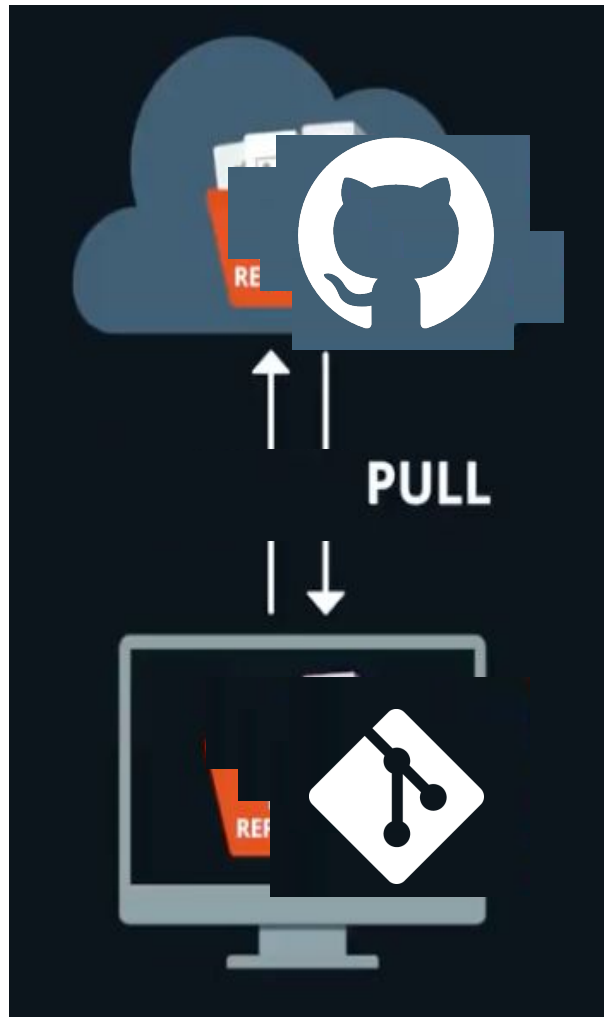


Git installation: <https://git-scm.com/install/>





Remote service



GitHub Account

1. Go to <https://github.com>.
2. Click on the “Sign up” at the right-top corner.
3. Enter your username of your choice (if it is already used, you will get some suggestions), email address, and password.
4. Follow further instruction and verify your account.

Git Configuration

Keep setup separate from the core concepts.

1 Verify Git

```
git --version
```

2 Identify yourself

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

Optional

Configure your preferred editor later; it is not required to understand today's workflow.

3 Set the default branch

```
git config --global init.defaultBranch  
main
```

A repository is a project plus its history

Git stores its metadata and history in a hidden `.git` directory.

my-project/

```
README.md  
analysis.R  
data_dictionary.csv  
.git/ ← Git metadata +  
history
```

Key idea

Your working files and Git's history live together as one repository.

Do not edit `.git` manually

Use Git commands to interact with the repository history.

- Linux/mac: `cd home/usesr/.../my-project`
- Windows : `cd C:/Users/user/.../my-project`

Two ways to get a Git repository

Choose based on where the project already exists.

I already have a local project

```
cd my-project  
git init  
git status
```

Initialize Git in that folder.

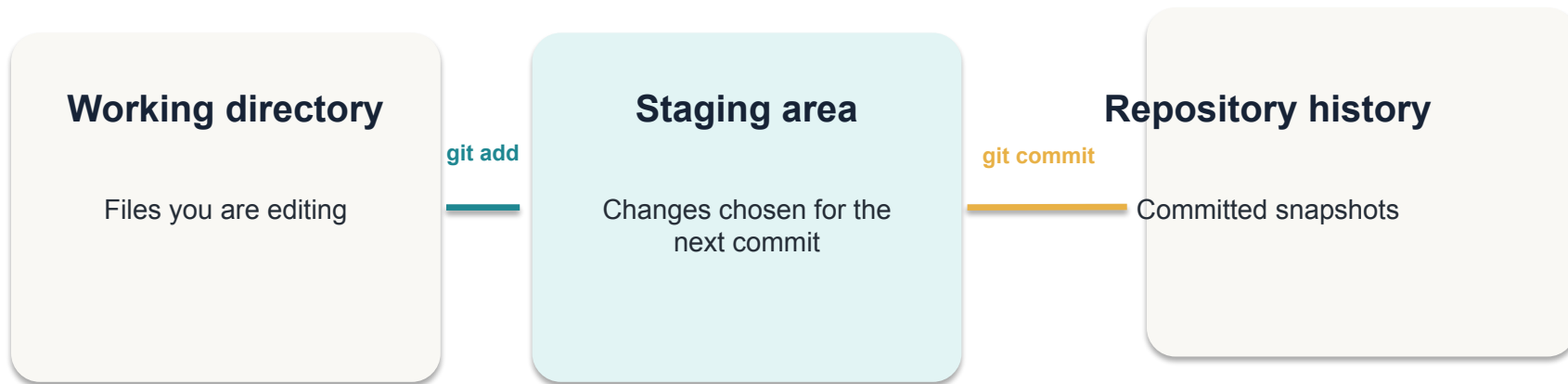
The repository already exists online

```
git clone <repository-url>  
cd <repository-folder>  
git status
```

Clone the repository and its history.

The core Git mental model

Changes move through three states before they become project history.



Editing a file is not the same as staging it; staging is not the same as committing it.

Three commands that make Git visible

When unsure, inspect before acting.

git status

Where am I?

Shows tracked, modified and staged files.

git diff

What changed?

Shows line-by-line changes not yet staged.

git log

What happened?

Shows committed project history.

Beginner habit: status → inspect → act → status again

Your first complete workflow

Make one small, meaningful change from start to finish.

1 Initialize

```
git init
```

2 Inspect

```
git status
```

3 Edit

```
README.md
```

4 Stage

```
git add README.md
```

5 Commit

```
git commit -m "Add project  
README"
```

6 Review

```
git log
```

A commit is a meaningful checkpoint

Good commits make your project history useful to your future self and collaborators.

Weak messages

```
update  
changes  
final changes  
stuff
```

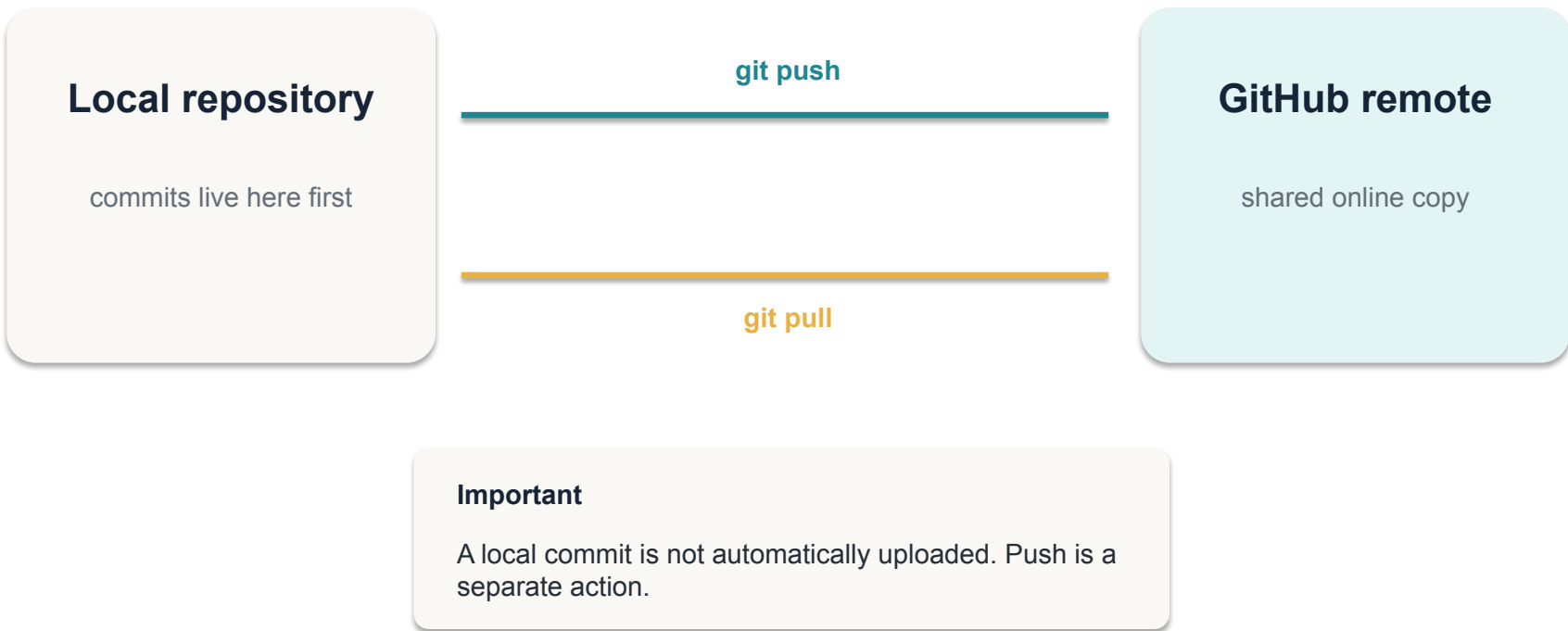
Useful messages

```
Add data-cleaning script  
Fix missing-value handling  
Update methods section  
Add regression diagnostic plot
```

Aim for one coherent change per commit.

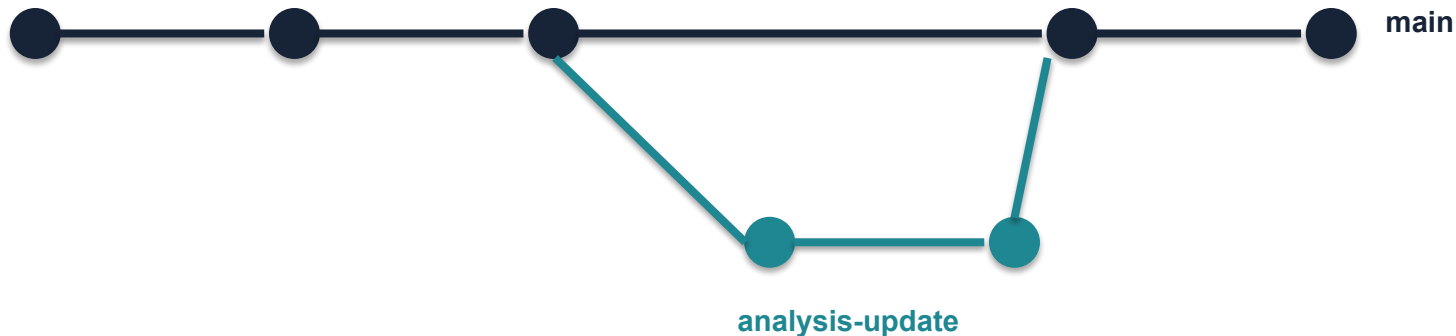
Connect local work to a remote repository

Commit locally; exchange commits through the remote.



Branches let you experiment safely

A branch is an independent line of development inside the same repository.



```
git switch -c analysis-update
```

A simple collaboration loop

Keep the first workflow small and repeatable.

1 Pull latest changes

2 Create a branch

3 Edit + commit

4 Push branch

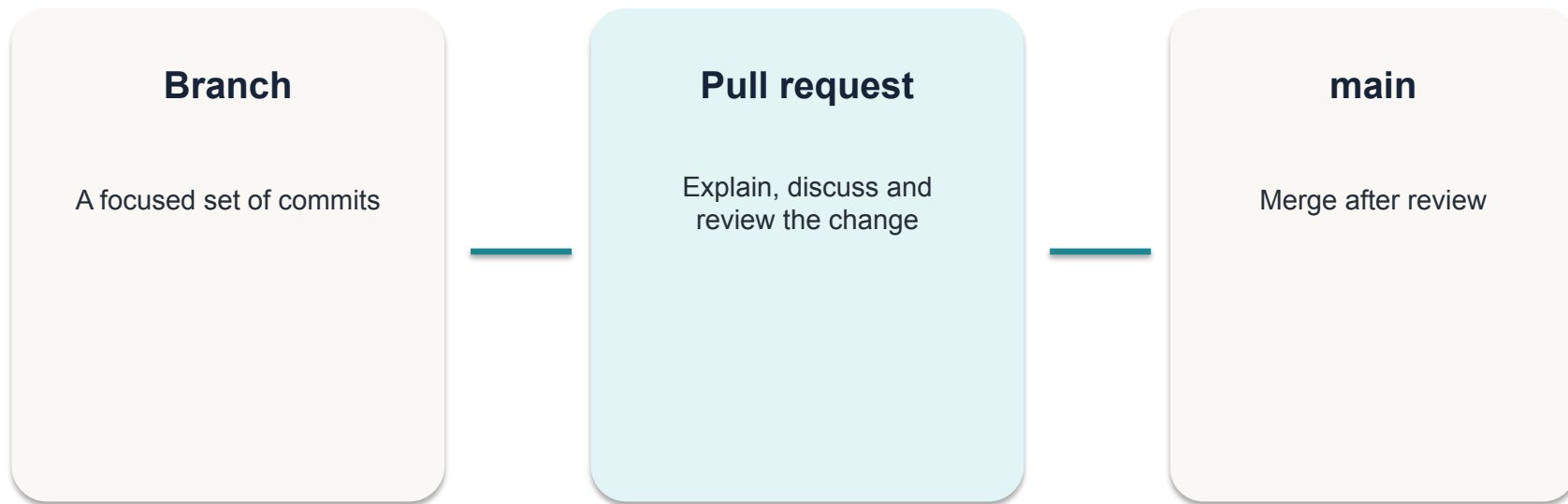
5 Open pull request

6 Review + merge

**Then collaborators pull the merged changes and
continue.**

Pull requests add a review step

Git tracks commits; GitHub helps teams discuss and merge them.



Merge conflicts: Git asks humans to decide

A conflict can occur when two changes overlap and Git cannot safely combine them.

```
<<<<<< HEAD
Benta's version
=====
Fatou's version
>>>>>> analysis-update
```

- Read both versions
- Choose or combine the intended content
- Remove the conflict markers
- Stage and commit the resolution

Key idea

Git manages history; it does not understand the meaning of your research.

Use .gitignore for files Git should not track

Keep the repository focused on source files and reproducible project inputs.

```
# Temporary files
*.tmp
*.log

# Generated output
results/
figures/

# Local secrets
.env
```

- Generated files that can be recreated
- Large or frequently changing binary files
- Machine-specific temporary files
- Secrets: passwords, tokens, API keys

Never commit secrets to a repository.

Research data: use judgment, not a blanket rule

Git is optimized for text and code; data management depends on size, sensitivity and workflow.

Often reasonable in Git

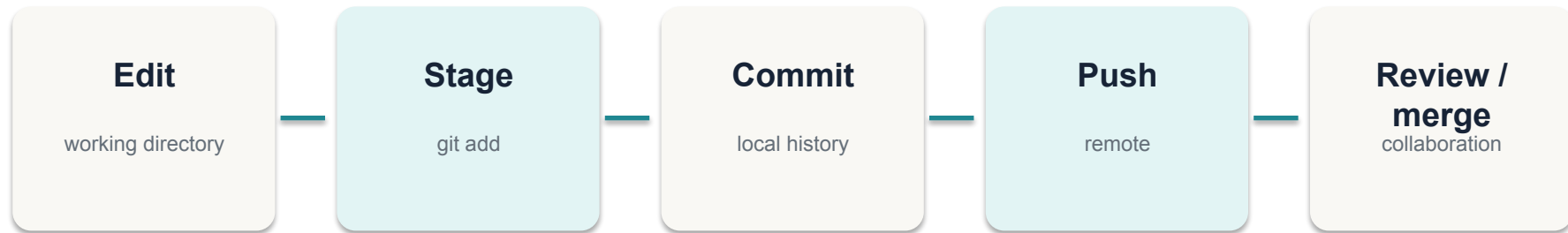
- Small, non-sensitive text datasets
- Metadata and data dictionaries
- Scripts that produce derived datasets
- Documentation of data provenance

Usually keep elsewhere

- Sensitive or restricted data
- Large binary datasets
- Raw data managed by a repository/archive
- Regenerable intermediate outputs

The complete beginner workflow

Keep this model in view while you practice.



Inspect at every stage with git status, git diff and git log.

Back to Benta and Fatou

Can you now describe a safer way for them to work on the report together?



- Clone or pull the shared repository
- Create separate branches for independent work
- Commit small, meaningful changes
- Push branches and review with pull requests
- Merge approved work and pull the updated main branch

Exit question

Explain the difference between edit, stage, commit and push in one sentence each.

Five things to remember

1

Git tracks versions; GitHub hosts and supports collaboration around repositories.

2

`git status` is your first diagnostic command.

3

Stage intentionally; commit meaningful changes.

4

Use branches to experiment without disrupting main.

5

Do not commit secrets or inappropriate data.

Next step: practice the workflow until the mental model feels routine.

Further reading & setup

Use these after class rather than crowding the core lecture.

Git installation

<https://git-scm.com/install>

Git documentation

<https://git-scm.com/doc>

Git visual learning

<https://learngitbranching.js.org/>

GitHub Docs: Get started

<https://docs.github.com/en/get-started>

PLOS: A Quick Introduction to Version Control with Git and GitHub

<https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004668>

Assignment:

Do this graded assignment

1- Clone this repo for SDS 711 course <https://github.com/Degoot-AM/SDS-711-Programming-for-data-science.git>

2- Add your name to “students_names.docx” in the Admin folder

3- Stage your edits, commit and push to the remote server

The repository will be updated weekly. Please pull the latest changes regularly to ensure that you have the most up-to-date course materials, including lectures, assignments, and announcements, on your local computer.